



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en Informática

Asignatura: Ingeniería del Software II

Unidad 4: Gestión de Riesgos y Deuda Técnica

Integrantes	C.i
Arnaldo Perdomo	30.791.551
Daniel Reyna	29.989.066
Jhoanny Maita	30.694.732
Yoryelis Ocando	31.074.651

Puerto Ordaz, 16 de Septiembre del 2026

Unidad 4: Gestión de Riesgos y Deuda Técnica

1. Considere Risk Storming y Riesgos Modernos Contexto: Una fintech está desarrollando una nueva aplicación móvil de préstamos que utiliza una API de Inteligencia Artificial de un tercero para evaluar el riesgo crediticio de los usuarios, y toda la infraestructura está alojada en un único proveedor de nube (Serverless).

Realice un Risk Storming simulado, identifique 4 riesgos específicos para este proyecto, uno de cada categoría: (a) Técnico/Deuda Técnica, (b) Ciberseguridad/Cadena de suministro, (c) Vendor Lock-in, (d) Ético o Legal (IA).

Para el riesgo de mayor puntuación, aplique un análisis cualitativo (asigne valores de Probabilidad 1-5 e Impacto 1-5) y calcule el Risk Score.

Proponga una estrategia de respuesta concreta (Evitar, Mitigar, Transferir o Aceptar) y describa cómo se traduciría esta mitigación en una tarea tangible dentro de un Sprint Backlog (ej. un Spike o una historia de seguridad).

Nos juntamos como equipo: desarrollo, QA, PO, alguien de seguridad y alguien de negocio. Dibujamos el flujo en una pizarra: usuario → app móvil → API Gateway → Lambda → DynamoDB → API externa de IA → decisión de crédito → notificación. A partir de ahí cada uno fue pegando riesgos en post-its. Eso es el Risk Storming: no lo hace solo el gerente, lo hace todo el equipo mirando la arquitectura y el flujo real.

Riesgos identificados

ID	Categoría	Riesgo detectado	Prob.	Impacto	Score	Estrategia
R1	Técnico / Deuda Técnica	Integrar la API de IA directo en el código, sin capa adaptadora ni pruebas de contrato. Si el proveedor cambia el JSON o la versión, toca reescribir media app.	4	4	16	Mitigar

R2	Ciberseguridad / Cadena de suministro	Depender de la API de IA y de librerías open source sin SBOM ni verificación de integridad. Además, se envían datos financieros sensibles a un tercero.	5	4	20	Mitigar / Transferir parcialmente
R3	Vendor Lock-in	Toda la infraestructura está en un solo cloud con servicios serverless propietarios: Lambda, DynamoDB, Cognito, Step Functions. Migrar después sale carísimo.	4	4	16	Mitigar

R4	Ético / Legal (IA)	El modelo de IA puede tener sesgos y rechazar préstamos a ciertos grupos por barrio, edad o género. Además, no es explicable y se envían datos personales sin base legal clara.	5	5	25	Mitigar
----	--------------------	---	---	---	----	---------

El riesgo con mayor puntuación fue el **R4: ético/legal por la IA**, con un Risk Score de **25**.

Análisis cualitativo del riesgo mayor

- **Probabilidad: 5.** Es casi seguro que si no se controla, aparezca algún sesgo o problema legal. El modelo es una caja negra, usa datos sensibles y la fintech no tiene control total sobre cómo fue entrenado.
- **Impacto: 5.** Puede generar multas, demandas, pérdida de licencia para operar, daño reputacional y exclusión financiera para usuarios.
- **Risk Score = $5 \times 5 = 25$.**

Estrategia de respuesta

La estrategia es **mitigar**. No se puede evitar del todo porque la IA es parte del negocio, pero tampoco se puede aceptar sin más. La fintech sigue siendo responsable legalmente, así que no basta con transferir todo al proveedor.

¿Cómo se traduce esto en tareas reales del Sprint Backlog?

- **Spike de 2 días:** evaluar sesgo y explicabilidad del modelo de IA. Entregable: informe con métricas de equidad, como paridad demográfica e igualdad de oportunidades, y recomendación de variables a excluir.
- **User Story de cumplimiento:** “Como analista de cumplimiento, quiero que la app pida consentimiento explícito y muestre una explicación simple de por qué se rechazó el crédito, para cumplir con normativa y dar derecho a apelación.”
- **Enabler técnico:** crear una capa de anonimización o tokenización antes de enviar datos a la API de IA.

- **Tarea de sprint:** agregar un botón de “Solicitar revisión humana” que cree un ticket y notifique a un analista.
- **Definition of Done:** incluir pruebas de sesgo, revisión legal y registro de auditoría.

Con esto, el riesgo no queda en un Excel muerto: se convierte en trabajo visible dentro del sprint.

2. **La organización requiere un plan de gestión de riesgos formal. Sin embargo, su equipo de desarrollo trabaja con Scrum y se niega a mantener un documento de Excel desactualizado.**

Diseñe un Registro de Riesgos (Risk Register) híbrido de una sola página que cumpla con los requisitos de auditoría (ID, Descripción, Probabilidad, Impacto, Dueño, Estado).

Explique el mecanismo de gobernanza para mantener este registro vivo: ¿Cómo se integra la revisión de riesgos en las ceremonias ágiles existentes ejemplos Sprint Planning y Sprint Retrospective) sin añadir burocracia innecesaria?

Dé un ejemplo de cómo un riesgo identificado como La API de IA podría cambiar sus términos de servicio y costos, se transforma en una User Story o Enabler priorizada en el Product Backlog.

El Registro de Riesgos (Risk Register) es el artefacto vivo más importante en la gestión de riesgos de un proyecto de software, ya que centraliza la descripción, probabilidad, impacto, dueño y estado de cada amenaza identificada (PMI, 2021). Sin embargo, en equipos ágiles como Scrum, un documento de Excel extenso y estático termina desactualizado y abandonado, generando fricción con la forma de trabajar del equipo. Por ello, se diseñó un **Risk Register híbrido de una sola página**: mantiene los campos mínimos exigidos por auditoría, pero en un formato lo suficientemente ligero como para vivir dentro del mismo tablero o backlog del equipo, sin convertirse en burocracia adicional.

A continuación se presenta el registro aplicado al caso de la fintech (app de préstamos con API de IA de terceros e infraestructura Serverless en un único proveedor cloud).

Tabla - Risk Register

ID	Descripción	Prob. (1-5)	Impacto (1-5)	Dueño (Risk Owner)	Estado
R-01	La API de IA de terceros podría cambiar sus términos de servicio y costos sin previo aviso.	4	4	Product Owner	Activo

R-02	Vendor lock-in: dependencia total de un único proveedor Serverless dificulta migración futura.	3	5	Arquitecto / Tech Lead	Activo
R-03	Vulnerabilidad heredada en dependencia open source usada por el módulo de scoring crediticio.	3	4	Equipo de Seguridad / QA	En monitoreo

Nota: R-01 y R-02 corresponden a los riesgos de mayor Risk Score (Probabilidad × Impacto = 16 y 15 respectivamente), por lo que son los candidatos prioritarios para convertirse en Spikes o Enablers dentro del Sprint Backlog.

Integración en ceremonias ágiles

La clave es que el Risk Register no se revisa aparte, sino que se cuelga de dos ceremonias que el equipo ya tiene agendadas:

En el Sprint Planning, antes de comprometer las historias del sprint, el equipo dedica unos minutos a mirar el registro y preguntarse si alguno de los riesgos activos debe convertirse en trabajo real de ese sprint: un Spike si hace falta investigar, o una historia habilitadora si ya se sabe qué hacer. También se revisa si las historias nuevas que van a entrar al sprint traen riesgos que aún no están anotados.

En la Sprint Retrospective, al cierre, el equipo actualiza el estado de los riesgos que se trabajaron (si se mitigaron, si siguen en monitoreo, si ya se cerraron) y anota cualquier riesgo nuevo que haya aparecido durante el sprint. Con esto el registro se mantiene vivo sin crear una reunión extra, simplemente se revisa como un punto más dentro de dos ceremonias que ya existen.

De riesgo a historia en el Product Backlog

El riesgo identificado como “La API de IA de terceros podría cambiar sus términos de servicio y costos sin previo aviso” se traduce en un Enabler o historia habilitadora, ya que busca garantizar que el sistema siga funcionando. A continuación, se presenta el formato estándar de una **User Story**, la plantilla que usa Scrum para escribir cualquier elemento del Product Backlog en la estructura “Como [rol], quiero [acción], para [beneficio]”:

Como equipo de desarrollo, **quiero** implementar un mecanismo de monitoreo y un plan de respaldo (fallback) para la integración con la API de scoring crediticio de terceros, **para** que la aplicación no dependa por completo de un único proveedor y el negocio no quede expuesto si cambian los términos o el costo sin aviso.

Criterios de aceptación posibles:

- El sistema registra alertas si la API cambia su versión, sus tiempos de respuesta o devuelve errores de forma sostenida.
- Se define un umbral de gasto por consulta y mensual que dispara alertas automáticas antes de sobrepasar el presupuesto operativo previsto.

Priorización en el Backlog:

Esta historia entra al Product Backlog con la misma prioridad que una funcionalidad normal, justificada por el Risk Score (Probabilidad × Impacto = 16, el más alto del registro), y compite por espacio en el sprint como cualquier otra historia.

3. Compare la efectividad de una sesión tradicional de identificación de riesgos (liderada únicamente por el Gerente de Proyectos con una lista de verificación) frente a una sesión de Risk Storming con todo el equipo multifuncional. ¿Por qué este último enfoque es más adecuado para detectar riesgos técnicos y de ciberseguridad en la arquitectura del software?

Comparación de los dos enfoques

Característica	Sesión tradicional (PM + checklist)	Risk Storming (equipo multifuncional)
Fuente de conocimiento	Única: el Gerente de Proyecto, según su experiencia y una lista predefinida; sin visibilidad directa del código, la arquitectura o las configuraciones reales.	Distribuida: cada rol aporta lo que ve desde su ángulo (el desarrollador conoce las dependencias del código, el QA los puntos de falla en pruebas, el PO el valor de negocio en riesgo).
Enfoque predominante	Riesgos de gestión (cronograma, presupuesto, alcance), por ser los que el PM percibe directamente.	Visual y colaborativo, normalmente sobre un diagrama de arquitectura o flujo de usuario, ubicando el riesgo en el componente exacto donde vive.
Punto débil / fuerte	Punto ciego ante riesgos técnicos y de ciberseguridad, que suelen estar ocultos en decisiones de implementación (una dependencia desactualizada, una API mal configurada, un login de terceros mal configurado).	Democratiza la detección de amenazas: nadie depende de que una sola persona recuerde todo.
Dinámica en el tiempo	Unidireccional y estática; el checklist no se actualiza con la realidad cambiante del sprint.	Iterativa; puede repetirse en cada sprint, ajustándose a cambios reales en la arquitectura.

¿Por qué el Risk Storming es más adecuado para riesgos técnicos y de ciberseguridad?

- **Proximidad al código: detección más precisa.** Los riesgos de ciberseguridad modernos (dependencias de terceros, servidores no actualizados, servicios de login mal configurados) son invisibles para alguien que no interactúa con el stack técnico día a día. Solo el equipo de desarrollo posee ese conocimiento tácito, por lo que su detección requiere su participación directa.
- **El diagrama de arquitectura como disparador cognitivo.** Al visualizar el flujo real del sistema sobre un diagrama de arquitectura, el equipo puede señalar puntos concretos de falla, por ejemplo una llamada a una API externa de IA sin validar su disponibilidad, o la ausencia de Zero Trust entre microservicios, algo

que una lista de verificación genérica no puede anticipar porque no fue diseñada para esa arquitectura específica.

- **Reducción del sesgo de autoridad única.** Una sesión liderada solo por el Gerente de Proyectos tiende a capturar los riesgos que él ya conoce o teme, como cronograma y costos, reforzando un sesgo de confirmación. El enfoque multifuncional expone riesgos incómodos que difícilmente alguien reportaría por iniciativa propia.
- **Alineación con la gestión ágil continua.** La gestión de riesgos moderna debe ser dinámica, continua y transversal, no un evento único al inicio del proyecto. El Risk Storming se integra naturalmente en ceremonias ágiles, como Sprint Planning y Retrospectiva, permitiendo detectar riesgos técnicos y de ciberseguridad de forma recurrente, mientras que el checklist tradicional es un evento aislado que rápidamente queda desactualizado.